

02_segmentation

September 27, 2026

1 02 — Shared example-start grid: reference-context candidate v2

Status: VERIFIED IMPLEMENTATION after all cells pass; candidate v2, not a frozen pair of frontends. The Core and exact source split remain FROZEN. Notebook 01 remains the independent dataset audit.

This notebook defines shared example identities and starts in whole-clip resampled 16 kHz audio. It does not implement either feature branch or train a model.

- Example start stride: **15,360 samples = 0.96 s**.
- Reference waveform context: **15,600 samples = 0.975 s**.
- Adjacent reference contexts overlap by **240 samples = 0.015 s**.
- Reference feature blocks begin **96 feature hops** apart.

96 complete 400-sample frames at hop 160 require $400 + 95 \cdot 160 = 15600$ samples. Thus non-overlapping groups of feature frames can have overlapping waveform support. No waveform completion padding is used; incomplete terminal reference contexts are discarded.

The old `core_096s_candidate_windows.csv` is preserved as historical evidence of the 15,360-context proposal. The new candidate uses a distinct filename and versioned IDs.

No RMS threshold, silence removal, manual cleaning, denoising, peak/RMS normalization, or activity gating. Technical validation fails on invalid inputs; it does not filter examples by signal activity.

```
[10]: from pathlib import Path
import hashlib, io, json, platform, wave
import numpy as np
import pandas as pd
import scipy
from scipy.io import wavfile
from scipy import signal

PROJECT = next((p for p in [Path.cwd(), *Path.cwd().parents]
                 if (p / "data/neurips_2021_zenodo_0_0_1.csv").is_file()), None)
assert PROJECT is not None, "Run from the project root or its notebook_
↳directory"
DATA_DIR = PROJECT / "data"
AUDIO_DIR = DATA_DIR / "audio"
```

```

SPLIT_PATH = DATA_DIR / "core_single_4species_frozen_split.csv"
OLD_MANIFEST = DATA_DIR / "core_096s_candidate_windows.csv"
OUTPUT_MANIFEST = DATA_DIR /
    ↪ "core_example_grid_v2_stride15360_ref15600_candidates.csv"
OUTPUT_AUDIT = DATA_DIR / "core_example_grid_v2_stride15360_ref15600_audit.json"

SOURCE_SR = 44100
TARGET_SR = 16000
RESAMPLE_UP = 160
RESAMPLE_DOWN = 441
EXAMPLE_STRIDE_SAMPLES = 15360
REFERENCE_CONTEXT_SAMPLES = 15600
REFERENCE_FRAME_SAMPLES = 400
REFERENCE_HOP_SAMPLES = 160
REFERENCE_FEATURE_FRAMES = 96
EXAMPLE_STRIDE_SECONDS = EXAMPLE_STRIDE_SAMPLES / TARGET_SR
REFERENCE_CONTEXT_SECONDS = REFERENCE_CONTEXT_SAMPLES / TARGET_SR
REFERENCE_CONTEXT_OVERLAP_SAMPLES = REFERENCE_CONTEXT_SAMPLES -
    ↪ EXAMPLE_STRIDE_SAMPLES
REFERENCE_CONTEXT_OVERLAP_SECONDS = REFERENCE_CONTEXT_OVERLAP_SAMPLES /
    ↪ TARGET_SR
assert SOURCE_SR * RESAMPLE_UP == TARGET_SR * RESAMPLE_DOWN

```

1.1 2.1 — Authenticate frozen inputs

These SHA-256 values pin the existing input bytes, not merely counts. A changed split must fail even if counts and leakage checks still pass. Do not update these hashes to accept another split. Hashes describe the reviewed files, not a new split recipe.

```

[11]: EXPECTED_INPUT_SHA256 = {'neurips_2021_zenodo_0_0_1.csv':
    ↪ 'baa4de3da5965dfea41dbd505023d392fab4f01db1e6e137cf50442423f79727',
    ↪ 'core_single_4species_metadata.csv':
    ↪ 'e72974da9a4e1a419bb459363be37ff91fdffd6e65ff9a32582e9de13e786614',
    ↪ 'core_single_4species_frozen_split.csv':
    ↪ '3e40928622dcea26e723540cfb493a35fd1d78393bf5531810583eae7966e0c4',
    ↪ 'core_audio_audit.csv':
    ↪ '8551ada9c64998e69922779b42054ec17245c8923457877d24b633813a1f2965',
    ↪ 'core_096s_candidate_windows.csv':
    ↪ '08df667e694466d23bd8c0a4ecb215f8505ea2e72c71cc131574bce4152628f9'}

def authenticate_inputs():
    for name, expected in EXPECTED_INPUT_SHA256.items():
        path = DATA_DIR / name
        assert path.is_file(), f"Missing frozen/historical input: {path}"
        assert hashlib.sha256(path.read_bytes()).hexdigest() == expected,
            ↪ f"Input identity mismatch: {name}"

```

```

authenticate_inputs()
split_df = pd.read_csv(SPLIT_PATH)
core_df = pd.read_csv(DATA_DIR / "core_single_4species_metadata.csv")
assert split_df[["id", "name", "species", "split", "length"]].notna().all().
↳all()
assert split_df["id"].is_unique and len(split_df) == 1898
assert split_df["name"].nunique() == 1189
assert split_df["split"].value_counts().to_dict() == {"train":1335,"validation":
↳269,"test":294}
assert split_df.groupby("split")["name"].nunique().to_dict() == {"train":
↳830,"validation":177,"test":182}
assert split_df.groupby("name")["split"].nunique().eq(1).all()
assert split_df.groupby("name")["species"].nunique().eq(1).all()
pd.testing.assert_frame_equal(
    split_df[core_df.columns].sort_values("id").reset_index(drop=True),
    core_df.sort_values("id").reset_index(drop=True), check_exact=True)
print("Authenticated frozen Core and exact parent split: PASS")

```

Authenticated frozen Core and exact parent split: PASS

1.2 2.2 — Visible VGGish geometry unit check

```

[12]: def complete_reference_frames(num_samples):
    return max(0, 1 + (num_samples - REFERENCE_FRAME_SAMPLES) //
↳REFERENCE_HOP_SAMPLES)

assert REFERENCE_CONTEXT_SAMPLES == REFERENCE_FRAME_SAMPLES +
↳(REFERENCE_FEATURE_FRAMES - 1) * REFERENCE_HOP_SAMPLES
assert complete_reference_frames(15600) == 96
assert complete_reference_frames(15360) == 94
assert EXAMPLE_STRIDE_SAMPLES == REFERENCE_FEATURE_FRAMES *
↳REFERENCE_HOP_SAMPLES
assert REFERENCE_CONTEXT_OVERLAP_SAMPLES == 240
print("15600 samples -> 96 complete frames; 15360 samples -> 94 complete
↳frames")
print(f"Start stride {EXAMPLE_STRIDE_SECONDS}s; reference context
↳{REFERENCE_CONTEXT_SECONDS}s; "
    f"adjacent context overlap {REFERENCE_CONTEXT_OVERLAP_SECONDS}s")

```

15600 samples -> 96 complete frames; 15360 samples -> 94 complete frames
Start stride 0.96s; reference context 0.975s; adjacent context overlap 0.015s

1.3 2.3 — Validated PCM24 loading and whole-clip resampling

SciPy returns PCM24 left-justified in int32: convert with float32 / 2**31. Header format and decoded dtype are checked before scaling. No amplitude normalization follows.

Resample the **whole clip** with up=160/down=441. Explicit arguments below match SciPy's de-

fault polyphase low-pass anti-alias filter (`window=('kaiser', 5.0)`) and zero boundary extension (`padtype='constant', cval=0`). Boundary extension inside filtering is distinct from waveform completion padding; no incomplete example is padded.

Expected output length is `ceil(native_samples*160/441)`, calculated using integer arithmetic. This SciPy path is a documented adaptation, not a claim of exact HumBugDB resampy/PCM16 parity.

```
[13]: def load_pcm24_float(wav_path):
    wav_path = Path(wav_path)
    if not wav_path.is_file():
        raise FileNotFoundError(wav_path)
    # Read once: the header check, decode, and provenance hash cover the same
    bytes.
    raw = wav_path.read_bytes()
    with wave.open(io.BytesIO(raw), "rb") as wf:
        sr, channels, width, frames = wf.getframerate(), wf.getnchannels(), wf.
        getsampwidth(), wf.getnframes()
        if (sr, channels, width, wf.getcomptype()) != (SOURCE_SR, 1, 3, "NONE"):
            raise ValueError(f"{wav_path.name}: expected 44100 Hz mono PCM24;
            got {sr}/{channels}/{width}")
        decoded_sr, audio = wavfile.read(io.BytesIO(raw))
        if decoded_sr != SOURCE_SR or audio.ndim != 1 or audio.dtype != np.
        dtype("int32"):
            raise ValueError(f"{wav_path.name}: unexpected decoded rate/shape/
            dtype")
        if len(audio) != frames or frames == 0:
            raise ValueError(f"{wav_path.name}: truncated or empty WAV")
        x = audio.astype(np.float32) / np.float32(2**31)
        if not np.isfinite(x).all():
            raise ValueError(f"{wav_path.name}: non-finite PCM conversion")
        return x, hashlib.sha256(raw).hexdigest()

def resample_to_16k(x):
    if x.ndim != 1 or not np.isfinite(x).all():
        raise ValueError("Invalid native waveform")
    y = signal.resample_poly(x, up=RESAMPLE_UP, down=RESAMPLE_DOWN,
                             window=('kaiser', 5.0), padtype="constant",
                             cval=0).astype(np.float32)
    expected = (len(x)*RESAMPLE_UP + RESAMPLE_DOWN - 1) // RESAMPLE_DOWN
    if len(y) != expected or not np.isfinite(y).all():
        raise ValueError("Non-finite or unexpected-length resampled output")
    return y

def get_example_starts(num_samples):
```

```

    return range(0, max(0, num_samples - REFERENCE_CONTEXT_SAMPLES + 1),
↳EXAMPLE_STRIDE_SAMPLES)

# Boundary cases: no hidden padding and correct terminal eligibility.
assert list(get_example_starts(15599)) == []
assert list(get_example_starts(15600)) == [0]
assert list(get_example_starts(30959)) == [0]
assert list(get_example_starts(30960)) == [0,15360]

```

1.4 2.4 — Generate and physically verify every reference context

All partitions receive only deterministic preprocessing and technical checks. No held-out signal diagnostics or parameter selection occur. Every emitted context is checked, not only a random sample.

```

[14]: manifest_rows, clip_records = [], []
for n, row in enumerate(split_df.itertuples(index=False), start=1):
    clip_id = int(row.id)
    x, audio_sha = load_pcm24_float(AUDIO_DIR / f"{clip_id}.wav")
    x16 = resample_to_16k(x)
    starts = list(get_example_starts(len(x16)))
    for example_index, start in enumerate(starts):
        end = start + REFERENCE_CONTEXT_SAMPLES
        context = x16[start:end]
        assert start >= 0 and end <= len(x16)
        assert len(context) == REFERENCE_CONTEXT_SAMPLES
        assert np.isfinite(context).all()
        manifest_rows.append({
            "example_uid": f"gridv2_{clip_id}_e{example_index:04d}",
            "id": clip_id, "name": row.name, "species": row.species, "split":
↳row.split,
            "example_index": example_index, "start_sample_16k": start,
            "start_second": start / TARGET_SR,
            "example_stride_samples": EXAMPLE_STRIDE_SAMPLES,
            "reference_context_samples": REFERENCE_CONTEXT_SAMPLES,
            "reference_end_sample_16k": end, "reference_end_second": end /
↳TARGET_SR,
            "target_sample_rate": TARGET_SR,
        })
    old_count = len(x16) // EXAMPLE_STRIDE_SAMPLES
    clip_records.append({"id":clip_id, "name":row.name, "species":row.
↳species,"split":row.split,
                        "native_samples":len(x), "resampled_samples":len(x16),
                        "old_context_candidates":old_count, "examples":
↳len(starts),
                        "terminal_examples_lost":old_count-len(starts),
↳"wav_sha256":audio_sha})

```

```

    if n % 250 == 0 or n == len(split_df):
        print(f"Validated {n}/{len(split_df)} clips")
seg = pd.DataFrame(manifest_rows)
clip_audit = pd.DataFrame(clip_records)
assert not seg.empty
print("Generated and physically validated examples:", len(seg))

```

```

Validated 250/1898 clips
Validated 500/1898 clips
Validated 750/1898 clips
Validated 1000/1898 clips
Validated 1250/1898 clips
Validated 1500/1898 clips
Validated 1750/1898 clips
Validated 1898/1898 clips
Generated and physically validated examples: 32645

```

1.5 2.5 — Exact parent, leakage, coverage, and geometry checks

```

[15]: assert seg.notna().all().all()
assert not seg["example_uid"].duplicated().any()
assert not seg.duplicated(["id", "start_sample_16k"]).any()
joined = seg.merge(split_df[["id", "name", "species", "split"]], on="id",
    ↳how="left",
    validate="many_to_one", suffixes=("", "_parent"),
    ↳indicator=True)
assert joined["_merge"].eq("both").all()
for field in ["name", "species", "split"]:
    assert joined[field].eq(joined[field+"_parent"]).all(), f"Parent mismatch: {field}"
assert seg.groupby("name")["split"].nunique().eq(1).all()
assert seg["start_sample_16k"].ge(0).all()
assert seg["start_sample_16k"].mod(EXAMPLE_STRIDE_SAMPLES).eq(0).all()
assert (seg["reference_end_sample_16k"]-seg["start_sample_16k"]).
    ↳eq(REFERENCE_CONTEXT_SAMPLES).all()
assert seg["example_stride_samples"].eq(EXAMPLE_STRIDE_SAMPLES).all()
assert seg["reference_context_samples"].eq(REFERENCE_CONTEXT_SAMPLES).all()
assert seg["target_sample_rate"].eq(TARGET_SR).all()
assert np.allclose(seg["start_second"], seg["start_sample_16k"] /
    ↳TARGET_SR, rtol=0, atol=1e-12)
assert np.allclose(seg["reference_end_second"], seg["reference_end_sample_16k"] /
    ↳TARGET_SR, rtol=0, atol=1e-12)
lengths = clip_audit.set_index("id")["resampled_samples"]
assert seg["reference_end_sample_16k"].le(seg["id"].map(lengths)).all()
ordered = seg.sort_values(["id", "example_index"])
assert ordered.groupby("id")["start_sample_16k"].first().eq(0).all()

```

```

assert ordered.groupby("id")["start_sample_16k"].diff().dropna().
    .eq(EXAMPLE_STRIDE_SAMPLES).all()
assert (ordered["example_index"] == ordered.groupby("id").cumcount()).all()
expected_uids = [f"gridv2_{r.id}_e{r.example_index:04d}" for r in seg.
    .itertuples(index=False)]
assert seg["example_uid"].tolist() == expected_uids
assert set(seg["id"]) == set(split_df["id"]), "Clip coverage changed;
    .investigate before promotion"
assert set(seg["name"]) == set(split_df["name"]), "Source coverage changed"
print("Exact parent inheritance, geometry, unique identity, coverage, zero
    .source leakage: PASS")

```

Exact parent inheritance, geometry, unique identity, coverage, zero source leakage: PASS

1.6 2.6 — Compare the preserved old candidate by parent/start coordinates

The expected old count is reconstructed from actual resampled lengths, then compared against the historical CSV. No candidate count is hard-coded. Only terminal examples lacking the extra 240 samples may disappear.

```

[16]: old_seg = pd.read_csv(OLD_MANIFEST)
assert not old_seg.duplicated(["id", "start_sample_16k"]).any()
old_parent = old_seg.
    .merge(split_df[["id", "name", "species", "split"]], on="id", how="left",
        .
    .validate="many_to_one", suffixes=("", "_parent"), indicator=True)
assert old_parent["_merge"].eq("both").all()
for field in ["name", "species", "split"]:
    assert old_parent[field].eq(old_parent[field+"_parent"]).all()
assert (old_seg["end_sample_16k"]-old_seg["start_sample_16k"]).
    .eq(EXAMPLE_STRIDE_SAMPLES).all()
expected_old_keys = {(int(r.id), s) for r in clip_audit.itertuples(index=False)
    for s in range(0, max(0, r.
    .resampled_samples-EXAMPLE_STRIDE_SAMPLES+1), EXAMPLE_STRIDE_SAMPLES)}
old_keys = set(zip(old_seg["id"], old_seg["start_sample_16k"]))
new_keys = set(zip(seg["id"], seg["start_sample_16k"]))
assert old_keys == expected_old_keys
assert new_keys <= old_keys
lost_keys = old_keys-new_keys
for clip_id, start in lost_keys:
    assert start+REFERENCE_CONTEXT_SAMPLES > lengths.loc[clip_id]
    assert start+EXAMPLE_STRIDE_SAMPLES <= lengths.loc[clip_id]
assert len(lost_keys) == int(clip_audit["terminal_examples_lost"].sum())
assert clip_audit["terminal_examples_lost"].isin([0, 1]).all()
print(f"Old candidates: {len(old_seg)}; new candidates: {len(seg)}; terminal
    .candidates lost: {len(lost_keys)}")

```

Old candidates: 32964; new candidates: 32645; terminal candidates lost: 319

1.7 2.7 — Coverage and count reports

Example counts are correlated observations, not independent biological samples. The sampling unit remains the source recording.

```
[17]: def coverage_summary(group):
    old_counts = old_seg.groupby(group).size().rename("old_examples")
    result = seg.groupby(group).
    ↪agg(examples=("example_uid", "size"), clips=("id", "nunique"), sources=("name", "nunique"))
    result = result.join(old_counts)
    result["terminal_examples_lost"] = result["old_examples"] - result["examples"]
    return result

split_summary = coverage_summary("split")
species_summary = coverage_summary("species")
split_species_summary = coverage_summary(["split", "species"])
per_clip = seg.groupby("id").size()
per_source = seg.groupby("name").size()
display(split_summary)
display(species_summary)
display(split_species_summary)
display(pd.DataFrame({"examples_per_clip": per_clip.
    ↪describe(), "examples_per_source": per_source.describe()}))
print(f"Clip coverage: {seg['id'].nunique()}/{len(split_df)}; source coverage:
    ↪{seg['name'].nunique()}/{split_df['name'].nunique()}")
```

	examples	clips	sources	old_examples	terminal_examples_lost
split					
test	5095	294	182	5146	51
train	22613	1335	830	22841	228
validation	4937	269	177	4977	40

	examples	clips	sources	old_examples \
species				
an arabiensis	14909	831	514	15057
an funestus ss	7487	381	248	7545
an squamosus	2092	141	91	2113
culex pipiens complex	8157	545	336	8249

	terminal_examples_lost
species	
an arabiensis	148
an funestus ss	58
an squamosus	21
culex pipiens complex	92

examples	clips	sources	old_examples \
----------	-------	---------	----------------

split	species				
test	an arabiensis	2232	124	78	2258
	an funestus ss	1239	61	38	1251
	an squamosus	371	23	15	372
	culex pipiens complex	1253	86	51	1265
train	an arabiensis	10457	588	359	10562
	an funestus ss	5128	267	173	5166
	an squamosus	1426	99	63	1442
	culex pipiens complex	5602	381	235	5671
validation	an arabiensis	2220	119	77	2237
	an funestus ss	1120	53	37	1128
	an squamosus	295	19	13	299
	culex pipiens complex	1302	78	50	1313

		terminal_examples_lost
split	species	
test	an arabiensis	26
	an funestus ss	12
	an squamosus	1
	culex pipiens complex	12
train	an arabiensis	105
	an funestus ss	38
	an squamosus	16
	culex pipiens complex	69
validation	an arabiensis	17
	an funestus ss	8
	an squamosus	4
	culex pipiens complex	11

	examples_per_clip	examples_per_source
count	1898.000000	1189.000000
mean	17.199684	27.455845
std	15.149498	14.580524
min	2.000000	2.000000
25%	5.000000	16.000000
50%	13.000000	29.000000
75%	29.000000	37.000000
max	122.000000	122.000000

Clip coverage: 1898/1898; source coverage: 1189/1189

1.8 2.8 — Future frontend contracts (not implemented here)

Reference Log-Mel: PROPOSED / NOT IMPLEMENTED. 16 kHz; 15,600-sample context; 96 frames; frame 400 samples first, hop 160, then FFT zero-padding to 512. Periodic Hann, no centered framing, no incomplete-frame padding; 64 filters over 125–7500 Hz; HTK-style Mel mapping with Mel-domain triangles, DC zero and no area normalization; unnormalized magnitude spectrum; natural $\log(\text{mel}+0.01)$. Do not assume `librosa.stft` reproduces these operations. Numerical frontend parity belongs to future 03A.

Edge MFE: PROPOSED / NOT FROZEN. Shared example IDs and start positions do not prescribe VGGish framing for Edge MFE. Its context, framing, frequency limits and noise floor remain undecided. Future paired comparisons must explicitly intersect branch eligibility while retaining these IDs. Do not interpret this manifest as a frozen Edge frontend or deployment contract.

The geometry/eligibility implementation can be frozen independently after review of its passing audits. This artifact retains candidate status so that frontend decisions are not implied.

1.9 2.9 — Save verified candidate and provenance report

Writes occur only after all preceding checks. Existing versioned outputs are compared rather than silently replaced. The report includes software versions, input hashes, per-clip audio hashes and lengths, coverage, and full physical-check counts. Canonical and historical inputs are authenticated again.

```
[18]: authenticate_inputs()
payload = seg.to_csv(index=False).encode("utf-8")
pd.testing.assert_frame_equal(seg,pd.read_csv(io.
    ↳BytesIO(payload)),check_dtype=False,
                                check_exact=False,rtol=0,atol=1e-12)
manifest_sha = hashlib.sha256(payload).hexdigest()
report = {
    "status":"VERIFIED_IMPLEMENTATION_CANDIDATE_NOT_FROZEN",
    "manifest":OUTPUT_MANIFEST.name, "manifest_sha256":manifest_sha,
    "software":{"python":platform.python_version(),"numpy":np.
    ↳__version__,"pandas":pd.__version__,"scipy":scipy.__version__},
    "input_sha256":EXPECTED_INPUT_SHA256,
    "geometry":{"sample_rate":TARGET_SR,"example_stride_samples":
    ↳EXAMPLE_STRIDE_SAMPLES,
                "reference_context_samples":
    ↳REFERENCE_CONTEXT_SAMPLES,"context_overlap_samples":
    ↳REFERENCE_CONTEXT_OVERLAP_SAMPLES,
                "reference_frame_samples":
    ↳REFERENCE_FRAME_SAMPLES,"reference_hop_samples":REFERENCE_HOP_SAMPLES,
                "reference_frames":REFERENCE_FEATURE_FRAMES,"padding":
    ↳"none","cleaning":"none"},
    "resampling":{"up":RESAMPLE_UP,"down":RESAMPLE_DOWN,"scope":
    ↳"whole_clip","window":["kaiser",5.0],
                "padtype":"constant","cval":0,"output_length":
    ↳"ceil(native_samples*160/441)"},
    "checks":{"all_assertions_passed":True,"physically_checked_contexts":
    ↳len(seg),
                "invalid_contexts":0,"source_leakage":0,"duplicate_ids":
    ↳0,"duplicate_geometry":0},
    "counts":{"old_examples":len(old_seg),"new_examples":
    ↳len(seg),"terminal_examples_lost":len(lost_keys),
```

```

        "clips":int(seg['id'].nunique()),"sources":int(seg['name'].
↪nunique())},
        "by_split":split_summary.reset_index().to_dict("records"),
        "by_species":species_summary.reset_index().to_dict("records"),
        "by_split_species":split_species_summary.reset_index().to_dict("records"),
        "examples_per_clip":per_clip.describe().to_dict(),"examples_per_source":
↪per_source.describe().to_dict(),
        "clips":clip_audit.to_dict("records"),
    }
    # Preflight both outputs before writing either one.
    if OUTPUT_MANIFEST.exists():
        assert OUTPUT_MANIFEST.read_bytes() == payload, "Existing v2 manifest_
↪differs; use a new version"
    if OUTPUT_AUDIT.exists():
        prior = json.loads(OUTPUT_AUDIT.read_text(encoding="utf-8"))
        assert prior["manifest_sha256"] == manifest_sha
        assert prior["input_sha256"] == EXPECTED_INPUT_SHA256
        assert prior["clips"] == report["clips"], "Audio identity/length changed;_
↪investigate"
    if not OUTPUT_MANIFEST.exists():
        with OUTPUT_MANIFEST.open("xb") as f: f.write(payload)
    if not OUTPUT_AUDIT.exists():
        with OUTPUT_AUDIT.open("x",encoding="utf-8") as f: json.
↪dump(report,f,indent=2,allow_nan=False)
    else:
        print("Existing matching audit preserved; current software versions:
↪",report["software"])
    authenticate_inputs()
    print("Saved/verified:",OUTPUT_MANIFEST)
    print("02_EXAMPLE_GRID: PASS - VERIFIED IMPLEMENTATION; candidate, not frozen_
↪frontends")

```

Existing matching audit preserved; current software versions: {'python': '3.11.9', 'numpy': '2.4.6', 'pandas': '3.0.6', 'scipy': '1.17.1'}

Saved/verified: d:\VGU-27\Edge AI

Project\data\core_example_grid_v2_stride15360_ref15600_candidates.csv

02_EXAMPLE_GRID: PASS - VERIFIED IMPLEMENTATION; candidate, not frozen frontends

1.10 FINAL CELL — Export notebook to PDF

```

[21]: from pathlib import Path
import subprocess
import sys

NOTEBOOK_PATH = PROJECT / "notebooks" / "current" / "02_segmentation.ipynb"
PDF_DIR = PROJECT / "reports" / "pdf"

```

```

PDF_DIR.mkdir(parents=True, exist_ok=True)

assert NOTEBOOK_PATH.exists(), f"Notebook not found: {NOTEBOOK_PATH}"

PDF_PATH = PDF_DIR / f"{NOTEBOOK_PATH.stem}.pdf"

print("Notebook :", NOTEBOOK_PATH)
print("PDF      :", PDF_PATH)
print("\nExporting with nbconvert + XeLaTeX...")

result = subprocess.run(
    [
        sys.executable,
        "-m",
        "jupyter",
        "nbconvert",
        "--to",
        "pdf",
        "--output",
        NOTEBOOK_PATH.stem,
        "--output-dir",
        str(PDF_DIR),
        str(NOTEBOOK_PATH),
    ],
    capture_output=True,
    text=True,
)

if result.returncode != 0:
    print("\n--- stdout ---")
    print(result.stdout)

    print("\n--- stderr ---")
    print(result.stderr)

    raise RuntimeError("PDF EXPORT: FAILED")

assert PDF_PATH.exists(), f"PDF was not created: {PDF_PATH}"

print("\n" + "=" * 70)
print("PDF EXPORT: PASS")
print("=" * 70)
print("Saved:", PDF_PATH)

```

Notebook : d:\VGU-27\Edge AI Project\notebooks\current\02_segmentation.ipynb
PDF : d:\VGU-27\Edge AI Project\reports\pdf\02_segmentation.pdf

Exporting with nbconvert + XeLaTeX...

=====

PDF EXPORT: PASS

=====

Saved: d:\VGU-27\Edge AI Project\reports\pdf\02_segmentation.pdf